Project Proposal: StockSync — Smart Inventory Management for a Two-Store Electronics Retailer

Team member	Student number	Primary responsibility
[Member A]	[Student number]	Backend API, PostgreSQL, transactions
[Member B]	[Student number]	Docker, Kubernetes, DigitalOcean, monitoring, CI/CD
[Member C]	[Student number]	React frontend, backups, integration testing

---

1. Motivation

Small electronics retailers with more than one store often rely on spreadsheets, informal messages, or disconnected inventory records. As a result, staff cannot always tell whether an item is available at their location, already sold, or being transferred. This is especially costly for high-value, theft-prone products such as laptops, phones, and gaming consoles.

StockSync addresses three concrete problems:

- **Overselling:** simultaneous transactions target the last unit, and both succeed.
- **Untracked transfers:** stock leaves one store but has not been confirmed at the other.
- **Limited accountability:** quantities change without an auditable record of who changed them, when, and why.

Intended users are store staff, who view availability and record sales or restocks; managers, who approve transfers and stock adjustments; and an administrator, who manages accounts and product information.

Why it matters: spreadsheets are inexpensive but do not reliably coordinate concurrent changes, while full retail suites can exceed the needs of a small two-store operator. A shared system with transactional stock updates and a searchable audit history can reduce discrepancies and improve coordination between stores.

This is also a meaningful cloud-computing problem. Inventory is **persistent state accessed concurrently by multiple users**, so the project is a focused setting for evaluating orchestration, storage persistence, scaling, security, and recovery, rather than simply displaying product records.

---

2. Objectives, Architecture, and Key Features

2.1 Objectives and application scope

We will build a web application for one fictional electronics retailer with two store locations. The minimum functional scope is:

- **Roles:** Administrator, Manager, and Staff, with permissions enforced on the server.
- **Products:** SKUs, names, categories, prices, and reorder thresholds.
- **Per-store stock:** quantities and availability at either location.
- **Sales and restocks:** stock decreases and increases that can never produce negative quantities.
- **Transfers:** request, dispatch, and receive inventory between stores.
- **Audit history:** action, quantity, product, store, user, and timestamp for every change.
- **Low-stock view:** stock below its configured reorder threshold.

We will seed realistic synthetic electronics inventory; integration with real point-of-sale systems is unnecessary. Suppliers, purchasing, invoicing, payments, and machine-learning forecasting are out of scope. The React interface will stay simple, because the course emphasizes cloud architecture and backend correctness.

2.2 Proposed architecture

Show Image

Figure 1: StockSync architecture. Users reach the cluster over HTTPS through a DigitalOcean Load Balancer and the managed Gateway API, which routes `/` to the React frontend and `/api` to the replicated Express API. PostgreSQL runs as a StatefulSet on a PVC, with nightly backups to Spaces.

The backend is a **stateless** Node.js/Express REST API; all persistent data lives in PostgreSQL 16. Any API replica can serve any request, and a Kubernetes Service distributes traffic across replicas.

For external traffic we will use **DigitalOcean's managed Gateway API** (Gateway and HTTPRoute resources), which is enabled by default on VPC-native DOKS clusters running Kubernetes 1.33 or later. We chose it over an NGINX Ingress controller because the community Ingress NGINX project was retired in March 2026. We will configure a domain and HTTPS; the

exact certificate mechanism will be validated on DOKS early. Locally on minikube, we will reach services through port-forwarding or NodePort.

2.3 Mandatory course technologies

Requirement	Implementation
Docker	Images for the React frontend and Express API; official PostgreSQL image.
Docker Compose	Reproducible local setup of API, frontend, and PostgreSQL, including a named development volume and seed data.
PostgreSQL	Tables for `users`, `stores`, `products`, `stock`, `transfers`, and `stock_movements`. Foreign keys, a unique `(store_id, product_id)` pair, and `CHECK (quantity >= 0)` protect integrity.
Persistent storage	Self-managed PostgreSQL **StatefulSet** (one replica) with a **PersistentVolumeClaim** provisioned by DigitalOcean `do-block-storage`. We do not substitute a managed database for the required storage implementation.
Cloud provider	DigitalOcean Kubernetes (DOKS), DigitalOcean Container Registry for images, and Spaces for database backups.
Orchestration	minikube locally; DOKS in the cloud. Deployments, Services, Gateway/HTTPRoute, StatefulSet, PVC, Secrets, CronJob, and HPA.
Monitoring	DOKS cluster and node metrics with threshold alerts (CPU, memory, disk), plus `kubectl top`, pod status, and application logs. We will confirm which metrics are available on our node pool before configuring alerts.

Inventory consistency. Each sale runs in a PostgreSQL transaction with an atomic conditional decrement:

```sql
UPDATE stock
SET quantity = quantity - $1
WHERE store_id = $2 AND product_id = $3 AND quantity >= $1
RETURNING quantity;
```

If no row is returned, the sale fails as out of stock. Successful changes and their audit entries commit together. Transfers have explicit *Requested* → *In Transit* → *Received* states: dispatch atomically removes units from the sending store and marks them in transit; receipt credits the

destination exactly once, and a transfer cannot be received twice. Cancellations use a compensating, audited transaction. Because correctness relies on database transactions rather than in-memory locks, it does not depend on which API replica handles a request.

2.4 Four committed advanced features

1. **Security enhancements:** bcrypt password hashing, JWT authentication, role-based authorization on API endpoints, HTTPS, and credentials stored in Kubernetes Secrets. We will verify that unauthorized roles cannot perform protected actions.
2. **Auto-scaling and high availability:** at least two API replicas for availability, and a Horizontal Pod Autoscaler that scales the API from 2 to at most 5 replicas based on CPU utilization. We will set CPU requests and verify the cluster's resource-metrics pipeline. Scaling depends on sustained load and available node capacity.
3. **CI/CD:** GitHub Actions runs automated tests on pull requests; successful merges build images, push them to DigitalOcean Container Registry, and deploy them to DOKS using securely stored credentials.
4. **Backup and recovery:** a nightly Kubernetes CronJob uses `pg_dump` to write compressed backups to DigitalOcean Spaces, with restricted credentials and a defined retention policy. We will restore a backup into a separate test database and verify stock and audit records without touching live data.

Optional, only after committed work passes validation: WebSocket stock updates, serverless notifications, and rule-based reorder suggestions.

2.5 Validation and tradeoffs

Test	Procedure	Expected evidence
Concurrent correctness	Parallel requests try to sell the final unit through replicated APIs	Exactly one sale succeeds; quantity is zero, not negative; matching audit entry
Scaling	Sustained k6 load that consumes API CPU	HPA raises replicas above two when thresholds and capacity permit; `kubectl get hpa -w`, pod counts, metrics
Persistence	Delete the PostgreSQL pod and let it be recreated	Products, quantities, and history remain on the PVC
Recovery	Restore a Spaces backup to a separate database	Record counts and sample records match the backup
Security and delivery	Attempt forbidden actions; push a change through CI/CD	Authorization failures; successful automated deployment

Correctness and scaling are tested separately: a short burst of requests proves transactional correctness but will not trigger the HPA, which reacts to CPU utilization measured over time.

Key choices and limitations. Kubernetes is more complex than Docker Swarm, but it provides native PVC integration and the HPA; minikube reduces early risk. Self-managed PostgreSQL fulfils the storage requirement, but a single database replica is a **single point of failure**: multiple API replicas improve API availability, not database availability. A PVC protects against pod replacement but is not a backup, and nightly backups could still lose up to about a day of changes. Gateway/TLS setup and HPA capacity are deployment risks that we will test early.

The planned scope is achievable for three people at roughly **125–170 total team-hours** (about 8–11 hours per member per week, budgeting 12 for buffer), with time reserved for integration and troubleshooting.

3. Project Plan and Team Responsibilities

Stage	Dates (2026)	Milestone
Foundation	Oct 30 – Nov 7	GitHub repo, proposal, API contract, schema, Docker Compose prototype
Core implementation	Nov 8 – 14	Authentication, products, stock changes, transfers, audit trail, React screens; minikube manifests
Cloud integration	Nov 15 – 20	DOKS deployment, database PVC, Gateway/HTTPS, monitoring, replica and persistence tests
Presentation readiness	Nov 21 – 24	HPA/load test, CI/CD, backup job, concurrency validation, demo rehearsal, slides
Final validation	Nov 25 – Dec 5	Restore test, bug fixes, README, AI interaction record, video, deployment check

Initial responsibilities

- **[Member A]:** Express API, schema and migrations, inventory transactions, API tests.
- **[Member B]:** Docker/Compose, minikube and DOKS manifests, Gateway/TLS, HPA, monitoring, CI/CD.
- **[Member C]:** React screens, backup/restore scripts, integrated tests, demo evidence.

All members review the architecture, test integrated changes, contribute documentation, and prepare for Q&A. GitHub branches, issues, pull requests, and meaningful commits record each member's contributions.

Integration and risks. We will define API and database contracts before parallel development, integrate at least weekly, and deploy a minimal API to DOKS early rather than postponing infrastructure work. The main risks are Kubernetes networking and storage setup, the HPA not scaling under realistic load, time pressure, and cloud cost. Mitigations include an early placeholder API deployment, explicit CPU requests and metrics checks, a fixed feature scope, and budget monitoring. The deployment will remain available during grading.

---

4. Initial Team Reasoning and AI-Assisted Exploration

> **[TEAM: complete 4.1 from your real initial discussion, verify 4.2, then delete these notes.]**

4.1 Initial team reasoning

Problem framing. Our first ideas were a car showroom app, a news app, and a school tracking app for teachers and parents; we then considered inventory management. [TEAM: describe what you initially saw as the problem and why inventory management seemed useful.]

Initial technical hypotheses. [TEAM: one or two approaches you genuinely considered before substantial AI assistance, and why. Do not claim an initial choice of Swarm or Kubernetes unless it happened.]

Open questions. [TEAM: real questions you raised, e.g. keeping data across container restarts, handling concurrent updates, Kubernetes versus Swarm, or deployment cost.]

4.2 Exploration and evaluation

We explored these questions through team discussion, the course rubric, Kubernetes and DigitalOcean documentation, and AI assistants.

Choosing the project. When AI-assisted exploration compared School Tracking with Smart Inventory Management, an early suggestion favoured School Tracking partly for its originality. Re-reading the rubric, which has no originality criterion and values depth over breadth, shifted our emphasis toward a focused system with demonstrable cloud architecture and transactional correctness. Given the November 24 demo cutoff, we selected inventory management as the more manageable option.

Testing autoscaling. An AI-assisted architecture discussion initially treated a concurrent last-item sale as evidence of both correctness and autoscaling. Further review against the Kubernetes HPA documentation showed that these need different tests: concurrency is validated by parallel transactions, while CPU-based HPA scaling needs sustained load and working resource metrics. We now test them separately. We also limited commitments to four advanced features, choosing backup and recovery over WebSockets because it complements persistent storage with a recoverability demonstration.

Checking current documentation. An early architecture draft used an NGINX Ingress controller. Checking current documentation showed that the community Ingress NGINX project was retired in March 2026 and that DOKS now provides a managed Gateway API by default, so we switched to Gateway and HTTPRoute resources.

We will continue to verify AI-generated implementation advice against documentation, test results, logs, and deployed behaviour rather than accepting it without evaluation.

Selected technical references: DOKS Gateway API documentation; DOKS metrics and monitoring; Kubernetes Horizontal Pod Autoscaling; Kubernetes StatefulSets; Kubernetes Ingress NGINX retirement notice (November 2025).